

TND FAIRFLOW ENGINE

Real-Time Intrinsic Valuation and Liquidity Pressure Model for USD/TND

Time Series Analysis

1. Abstract

The TND FairFlow Engine is a two-layer daily-frequency model for estimating the intrinsic value of the USD/TND exchange rate. The first layer regresses log returns of the official BCT USD/TND fixing on log returns of EUR/USD, GBP/USD, and USD/JPY, using ordinary least squares with Newey-West heteroskedasticity- and autocorrelation-consistent standard errors. The resulting coefficient estimates are applied via a compounding formula to reconstruct a global basket baseline from the previous fixing level. A parallel rolling-window version of the same regression produces time-varying coefficients used as the primary baseline in production. The second layer models the interbank-fixing spread as a function of its own autoregressive lags, a twenty-day rolling mean, a twenty-day rolling standard deviation, a lagged one-dimensional MLE-calibrated Kalman state, and a high-stress regime indicator. The Kalman filter parameters Q and R are estimated by maximum likelihood using Nelder-Mead optimization of the negative log-likelihood of the local-level state-space model. A data-driven anchor selection procedure compares three spread definitions — current fixing, lagged fixing, and EUR/TND cross-implied — using AIC and residual standard deviation, and automatically selects the best-fit anchor. The two layers are combined additively into a final intrinsic value estimate. Rolling out-of-sample backtesting against the next-day realized interbank rate evaluates RMSE, MAE, MAPE, and directional accuracy relative to a fixing-only benchmark. Forecast efficiency is assessed through the Diebold-Mariano test and the Mincer-Zarnowitz regression. Regime-conditional performance is computed separately for high-stress and normal-stress periods as classified by the 75th percentile of rolling spread volatility.

2. Introduction

The official USD/TND exchange rate is published by the Central Bank of Tunisia as a twice-daily fixing benchmark. In practice, the interbank market rate at which commercial banks transact USD/TND deviates from this benchmark because of local liquidity conditions, foreign-currency

scarcity, balance-sheet constraints, and temporary supply-demand imbalances. These deviations are not captured by the official fixing alone.

The TND FairFlow Engine addresses this gap by constructing a real-time intrinsic value estimate for USD/TND that combines two empirically estimated components. The global basket component produces a fundamental anchor derived from international currency movements. The local liquidity adjustment component estimates the spread between the interbank rate and the fixing, conditioned on its own persistence, its recent volatility regime, and a latent Kalman-filtered state variable. The two components are combined additively.

The model is implemented as a daily-frequency prototype in Python. The architecture is designed to accept higher-frequency global FX inputs in a live deployment, with the official BCT fixing serving as the anchor at each update cycle. The present notebook documents all estimation steps, diagnostic tests, backtesting procedures, and output constructions that constitute the full deliverable.

3. Data Description

3.1 Input Files and Variables

The dataset is constructed from four source files. The first contains the official BCT USD/TND fixing mid-price with a date column labeled Exchange Date and a value column labeled Mid Price. The second contains the official BCT EUR/TND fixing mid-price in the same format. The third contains the Tunisian interbank USD/TND mid-price with a Date column and a USD column. The fourth contains the global FX basket variables EUR/USD, GBP/USD, and USD/JPY identified by column-name keyword matching.

3.2 Data Cleaning and Alignment

All four datasets are parsed using a coerce-safe date parser, sorted chronologically, and deduplicated on date before merging. The merge is performed as a sequential inner join on the exchange date across all four sources. After merging, the panel is sorted and the index is reset. The final aligned dataset is daily and covers the common date range across all four sources. Missing values in any of the modeled variables result in row exclusion at the feature-construction stage.

3.3 Feature Engineering

The following derived variables are computed on the merged panel. Log returns are computed as first differences of natural logarithms for USD/TND fixing, interbank USD/TND, EUR/TND fixing, EUR/USD, GBP/USD, and USD/JPY. Percentage changes are computed for USD/TND fixing, EUR/USD, GBP/USD, and USD/JPY. The interbank-fixing spread is defined as:

$$\text{Spread}_t = \text{usd_tnd_ib_t} - \text{usd_tnd_fix_t}$$

The spread in basis points is defined as:

$$\text{Spread_bps}_t = 10000 \times \text{Spread}_t / \text{usd_tnd_fix_t}$$

Lagged spread values at lags one and two, a twenty-day rolling mean of the spread, and a twenty-day rolling standard deviation of the spread are also constructed. The variable `actual_next_ib` is the interbank rate shifted one step backward in time, representing the next-day realized interbank outcome used as the backtest target.

4. Model Framework

The intrinsic value of USD/TND is defined as the additive combination of two estimated components:

$$\text{Intrinsic}_t = \text{Baseline}_t + \text{Adjustment}_t$$

Baseline_t is the fundamental USD/TND level implied by the previous official fixing compounded by the basket-predicted log return. Adjustment_t is the estimated local liquidity deviation predicted by the autoregressive liquidity regression. The two components are estimated independently from distinct model objects and combined at the construction stage.

The model is trained on the first seventy-five percent of the aligned sample. Out-of-sample performance is evaluated on the remaining observations using a rolling-window re-estimation procedure that prevents any look-ahead from future spread or basket observations.

5. Global Basket Model

5.1 Static Log-Return Regression

The primary basket specification regresses the log return of the official USD/TND fixing on the contemporaneous log returns of EUR/USD, GBP/USD, and USD/JPY. The estimation uses ordinary least squares via statsmodels OLS with Newey-West HAC-robust standard errors. The HAC covariance is applied with a maximum lag of five to account for potential serial correlation and heteroskedasticity in FX return residuals. The regression is estimated on the training subsample defined as the first seventy-five percent of df_model. The fitted model object is basket_log_model. The specification is:

$$r_usd_tnd_fix_t = \beta_0 + \beta_1 * r_eurusd_t + \beta_2 * r_gbpusd_t + \beta_3 * r_usdjpy_t + \epsilon_t$$

where all variables are log returns. Predictions from the static model are stored in basket_return_static. The full basket_log_model.summary() output is displayed in the notebook.

5.2 Benchmark Percentage-Change Regression

A parallel benchmark regression is estimated using simple percentage changes instead of log returns. The dependent variable is pct_usd_tnd_fix and the regressors are pct_eurusd, pct_gbpusd, and pct_usdjpy. Estimation uses the same OLS + HAC procedure with maxlags equal to five, applied to the first seventy-five percent of the percentage-change sample. This model is labeled pct_model and is used for comparison purposes only. Its predictions are not incorporated into the intrinsic value output.

5.3 Rolling Coefficient Estimation

To allow basket sensitivities to evolve over time, a rolling OLS procedure re-estimates the three coefficients at each observation using a trailing window. The window length is set to the minimum of one hundred twenty and the maximum of fifty and one-third of the available sample, subject to being at least fifty observations. For each rolling window ending at observation *i*, a new OLS model is fitted on the window data using standard OLS without HAC correction. The prediction for observation *i* is generated from that window model and stored alongside the associated beta

estimates for EUR/USD, GBP/USD, USD/JPY, and the in-window R-squared. The rolling predictions are merged into `df_model` as `basket_return_rolling`. The final field `basket_return_final` fills rolling predictions where available and falls back to the static predictions elsewhere.

6. Baseline Reconstruction

The global basket baseline is defined by compounding the previous official fixing by the predicted basket log return. For the static model:

```
baseline_static_t = usd_tnd_fix_(t-1) * exp(basket_return_static_t)
```

For the rolling model:

```
baseline_rolling_t = usd_tnd_fix_(t-1) * exp(basket_return_final_t)
```

The rolling baseline is the primary baseline used in the intrinsic value construction and the backtest. This formulation ensures that the baseline level is anchored at each step to the most recently observed official fixing, making it suitable for deployment as a real-time intraday engine once global FX updates are fed continuously. Both baseline series are stored in `df_model`.

7. Liquidity Adjustment Model

7.1 Fixing Anchor Selection

The project brief identifies a research question regarding which fixing anchor best characterizes the liquidity spread. Three candidate spread definitions are evaluated in the anchor sensitivity analysis. The first is the spread against the current-day fixing (`usd_tnd_ib` minus `usd_tnd_fix`), labeled `spread_vs_current`. The second is the spread against the previous-day fixing (`usd_tnd_ib` minus `usd_tnd_fix` shifted by one), labeled `spread_vs_lag1`. The third is the spread against a cross-implied USD/TND anchor derived as `eur_tnd_fix` divided by `eurusd`, labeled `spread_vs_cross_anchor`.

For each candidate, a LinearRegression model is fitted using four features: lag-one spread, lag-two spread, twenty-day rolling mean of spread, and twenty-day rolling standard deviation of spread. The fitted models are evaluated on AIC, residual standard deviation, and R-squared. The anchor with the lowest AIC (primary criterion), lowest residual standard deviation (secondary), and highest R-squared (tertiary) is selected automatically as `selected_anchor_name`. The working spread column in `df_model` is then replaced with the selected anchor definition for all downstream steps.

7.2 Kalman Filter with MLE Calibration

A one-dimensional local-level Kalman filter is applied to the selected-anchor spread series to extract a smooth latent liquidity-pressure state. The filter equations are:

$$\begin{aligned} \mathbf{x}_t &= \mathbf{x}_{(t-1)} + \mathbf{w}_t & \mathbf{w}_t &\sim \mathbf{N}(0, \mathbf{Q}) \\ \mathbf{y}_t &= \mathbf{x}_t + \mathbf{v}_t & \mathbf{v}_t &\sim \mathbf{N}(0, \mathbf{R}) \end{aligned}$$

where $\mathbf{y}_t$ is the observed spread and $\mathbf{x}_t$ is the unobserved latent liquidity state. The parameters $\mathbf{Q}$ (state transition variance) and $\mathbf{R}$ (observation variance) are estimated by maximum likelihood rather than set heuristically. The MLE procedure minimizes the negative log-likelihood of the

Kalman filter applied to the full selected-anchor spread series. Optimization is performed using the Nelder-Mead method via `scipy.optimize.minimize` with a maximum of five thousand iterations. Initial values are $Q = 0.05 * \text{var}(\text{spread})$ and $R = 0.50 * \text{var}(\text{spread})$. Optimization is conducted in log-parameter space to enforce positivity: $\log_Q$ and $\log_R$ are the optimization variables, and Q_mle and R_mle are their exponentials.

The calibrated Kalman state series is stored as `kalman_spread_mle` (from the MLE calibration cell) and `kalman_spread_insample` (from the main filter application cell). The posterior variance at each step is stored as `kalman_var_mle`. The lagged Kalman state `kalman_lag1` is used as a predictor in the liquidity regression.

7.3 Regime Classification

A high-stress regime indicator is constructed from the twenty-day rolling standard deviation of the selected spread. Observations are classified as High Stress when the rolling standard deviation exceeds the 75th percentile of the full rolling standard deviation series; otherwise they are classified as Normal. This produces a binary regime column in `df_model`. Summary statistics for average spread, spread volatility, and average spread in basis points are computed separately by regime. The binary high-stress dummy (`high_stress_dummy`) is used as a feature in the liquidity adjustment regression.

7.4 Liquidity Adjustment Regression

The liquidity adjustment is estimated by a sklearn LinearRegression model using six features: `spread_lag1`, `spread_lag2`, `spread_roll_mean_20`, `spread_roll_vol_20`, `kalman_lag1`, and `high_stress_dummy`. The target variable is the selected-anchor spread. Training uses the first seventy-five percent of the rows remaining after constructing all six features, with no intercept suppression. The model is fitted as `liq_model`. In-sample predictions are stored as `adjustment_estimate_insample` in `df_model`.

8. Intrinsic Value Construction

8.1 Point Estimate

The final intrinsic value is the additive combination of the rolling basket baseline and the in-sample liquidity adjustment:

```
intrinsic_insample_t = baseline_rolling_t +  
                        adjustment_estimate_insample_t
```

8.2 Confidence Bands

Two sets of confidence bands are constructed. The Kalman posterior band is centered on `intrinsic_insample` and uses 1.96 times the square root of `kalman_var_mle` as the half-width:

```
intrinsic_upper_t = intrinsic_insample_t + 1.96 *  
                    sqrt(kalman_var_mle_t)  
  
intrinsic_lower_t = intrinsic_insample_t - 1.96 *  
                    sqrt(kalman_var_mle_t)
```

The empirical band is constructed from the sixty-day rolling standard deviation of the residual series (`usd_tnd_ib` minus `intrinsic_insample`), labeled `resid_vol_60`, scaled by 1.96:

```
intrinsic_empirical_upper_t = intrinsic_insample_t + 1.96 *  
                             resid_vol_60_t  
intrinsic_empirical_lower_t = intrinsic_insample_t - 1.96 *  
                             resid_vol_60_t
```

8.3 Market Pressure Gauge

The market pressure gauge is a real-time monitoring output computed from the final row of `df_model` with non-null intrinsic value. It reports the official fixing, interbank rate, basket baseline, liquidity adjustment, intrinsic value, and the market gap defined as `usd_tnd_ib` minus `intrinsic_insample`. A liquidity stress z-score is computed as (spread minus mean spread) divided by standard deviation of spread over the full available history. A positive market gap indicates that USD/TND is trading above intrinsic value, consistent with local USD scarcity or liquidity stress.

8.4 Nowcast Function

A production-style function `fairflow_nowcast` is defined to produce one-step intrinsic value estimates from live inputs. The function accepts the latest available BCT fixing, current and previous global FX levels, the fitted basket model, the fitted liquidity model, and the latest liquidity state features. It computes log returns from current and previous FX levels, passes them through the basket model with a constant term, exponentiates the predicted return compounded from the latest fixing to obtain the baseline, then adds the liquidity adjustment predicted by the linear liquidity model. The function returns the expected basket return, the baseline, the estimated adjustment, and the intrinsic USD/TND value.

9. Backtesting Framework

The out-of-sample backtest avoids look-ahead bias by re-estimating both the basket and liquidity models from scratch at each step using only information available up to the forecast date. A minimum training window `MIN_TRAIN` is set to the minimum of one hundred twenty and the maximum of sixty and forty-five percent of the available sample. Backtest steps are executed every five observations to reduce computation time.

At each step i , the historical slice up to i is cleaned to retain rows with complete basket and spread variables. A new sklearn `LinearRegression` basket model is fitted on the historical basket returns without a constant. A Kalman filter is applied to the historical spread using the global `Q_mle` and `R_mle` parameters estimated on the full sample. The final Kalman state from the historical filter is used as the lagged Kalman predictor for the current liquidity feature vector. A new liquidity model is fitted on the historical data using the same six-feature structure, with rolling statistics and regime indicators recomputed from the historical slice only. The one-step-ahead intrinsic value forecast and the fixing-only benchmark are recorded alongside the realized next-day interbank rate.

10. Performance Evaluation

10.1 Aggregate Metrics

Forecast performance is evaluated using four metrics computed on the backtest series `bt`. RMSE is the square root of mean squared error between the actual next-day interbank rate and the

forecast. MAE is the mean absolute error. MAPE is the mean absolute percentage error expressed as a percentage, computed only over rows where the actual value exceeds a threshold of 1e-12 to avoid division by near-zero values. Directional accuracy is the proportion of backtest steps at which the sign of the change in the intrinsic forecast matches the sign of the change in the actual interbank rate, computed over the full backtest series with a one-step shift applied to both series. These four metrics are computed for both the FairFlow intrinsic engine and the fixing-only benchmark and presented in the metrics table.

10.2 Regime-Conditional Performance

RMSE and MAE are computed separately for each regime (High Stress and Normal) for both the FairFlow model and the fixing-only benchmark. RMSE improvement and MAE improvement are computed as fixing metric minus FairFlow metric, so that positive values indicate FairFlow outperformance. A conditional interpretation is produced: if the RMSE improvement in the High Stress regime is positive, the liquidity adjustment is confirmed to add value during stress periods.

11. Statistical Validation

11.1 Stationarity Tests

Seven series are tested for stationarity: USD/TND fixing level, interbank USD/TND level, liquidity spread, USD/TND fixing log return, EUR/USD log return, GBP/USD log return, and USD/JPY log return. Each series is passed through the Augmented Dickey-Fuller test with AIC-based lag selection and the KPSS test with automatic lag selection and a constant specification. The ADF null is a unit root; the KPSS null is stationarity. A series is classified as Stationary if ADF p-value is below 0.05 and KPSS p-value is above 0.05. It is classified as Likely non-stationary if ADF p-value is at or above 0.05 and KPSS p-value is at or below 0.05. Otherwise, it is classified as Mixed evidence. Results are tabulated as `stationarity_results` and exported to the Excel deliverable.

11.2 Residual Diagnostics

The residuals from the static basket regression are tested using the Ljung-Box autocorrelation test at lags five, ten, and twenty, and the ARCH LM test with ten lags from statsmodels. The residual series is plotted over time and the autocorrelation function is plotted up to a maximum of thirty lags or one-third of the residual length, whichever is smaller. These diagnostics are reported as `ljung_box` and `arch_results` and are exported to the Excel deliverable.

11.3 Diebold-Mariano Test

A Diebold-Mariano test formally compares the forecast loss differential between the fixing-only benchmark and the FairFlow intrinsic engine. The loss function is squared error (power equal to two). The test statistic is constructed using a variance estimator that accounts for forecast-horizon autocorrelation up to order h equal to one. The p-value is computed from a two-sided t-distribution with degrees of freedom equal to the number of aligned observations minus one. The result is interpreted as: FairFlow improvement statistically significant at 5% if the p-value is below 0.05 and the DM statistic is positive, or No statistically significant FairFlow improvement at 5% otherwise.

11.4 Mincer-Zarnowitz Regression

Forecast efficiency is evaluated by regressing the actual next-day interbank rate on a constant and the FairFlow intrinsic forecast using OLS with Newey-West HAC standard errors at maxlags equal to one. The joint null hypothesis is:

$$H_0: \alpha = 0 \text{ AND } \beta = 1$$

This null is tested using a Wald F-test of the joint restriction $\text{const} = 0$, $\text{intrinsic_oos} = 1$. The interpretation is that the null not rejected indicates the forecast is statistically consistent with unbiased and efficient prediction. If the null is rejected with β below one, the forecast is interpreted as overreacting to variation. If the null is rejected with β above one, the forecast is interpreted as underreacting. The result includes the intercept estimate α , slope estimate β , in-sample R-squared, joint F-statistic, and joint p-value.

12. Regime Analysis

Regime classification is implemented through the rolling spread volatility indicator described in Section 7.3. The twenty-day rolling standard deviation of the selected spread is compared against its 75th-percentile threshold. Observations above this threshold are assigned to the High Stress regime; all others are assigned to Normal. This produces a binary categorical variable regime appended to `df_model`.

Regime statistics are computed using a grouped aggregation that reports observation count, mean spread, spread standard deviation, and mean spread in basis points for each regime. These statistics are exported to the backtest Excel sheet. The regime label is carried into the backtest loop so that performance metrics can be disaggregated by regime in the `regime_backtest` table.

In the intrinsic value output, the `high_stress_dummy` enters the liquidity regression as a binary regressor. Its estimated coefficient captures the incremental adjustment to the spread prediction that the model attributes to high-stress market conditions, holding the autoregressive and rolling-moment features constant.

13. Conclusion

The TND FairFlow Engine constructs a daily intrinsic value for USD/TND by combining a global FX basket baseline with a data-driven local liquidity adjustment. The basket component is estimated as a log-return OLS regression of the official BCT fixing on EUR/USD, GBP/USD, and USD/JPY log returns, using HAC-robust standard errors and a rolling re-estimation procedure for coefficient stability. The baseline is recovered by compounding the previous fixing with the predicted basket return.

The local liquidity component is estimated from the interbank-fixing spread. An anchor selection procedure evaluates three spread definitions using AIC and automatically selects the most statistically stable basis for the adjustment. A one-dimensional local-level Kalman filter with MLE-calibrated Q and R parameters extracts a smooth latent liquidity-pressure state. The adjustment regression combines spread lags, rolling moments, the lagged Kalman state, and a high-stress dummy into a linear prediction of the current spread.

Validation is performed using rolling out-of-sample backtesting against the realized next-day interbank rate, with RMSE, MAE, MAPE, and directional accuracy evaluated relative to a fixing-only benchmark. The Diebold-Mariano test provides a formal statistical comparison of

forecast accuracy. The Mincer-Zarnowitz regression tests whether the intrinsic forecast is unbiased and efficient. Stationarity tests on the input series confirm the appropriateness of a return-based specification. Ljung-Box and ARCH diagnostics characterize the residual structure of the basket model.

The complete framework is implemented in Python using statsmodels, sklearn, scipy, and numpy, with all intermediate outputs exported to a structured Excel deliverable containing seventeen sheets covering raw data, model results, diagnostics, backtesting, and regime analysis.